SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE NAME: Artificial Intelligence COURSE CODE: CSA17

COURSE OUTCOME COVERED

CO5: *Design AI-based systems using PySpark, RDD operations, and intelligent agent architectures, using scalable systems and integrate components in distributed AI applications.* (BTL 5)

Assessment Tool Weightage: 50%

QUESTIONS: 5

Duration : 1 Hour
Total Marks:50

Annexure A

Design Task

Code of Conduct:

I _M KAVYA 192472370_ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:KAVYA

Annexure A

Design Task

1. **Hybrid AI Recommendation Engine Design**

   Design a hybrid AI-based recommendation system that integrates content-based and collaborative filtering techniques using PySpark RDDs. Explain how distributed data processing can be used to handle large-scale user-item interactions, and describe the role of intelligent agents in adapting recommendations based on user behavior and feedback in real time.

1.1 Problem Understanding and Requirement Analysis

A Hybrid AI Recommendation Engine is designed to provide personalized recommendations by combining **Content-Based Filtering** and **Collaborative Filtering**. The system analyzes large-scale user-item interaction data such as ratings, purchases, clicks, searches, and browsing history.

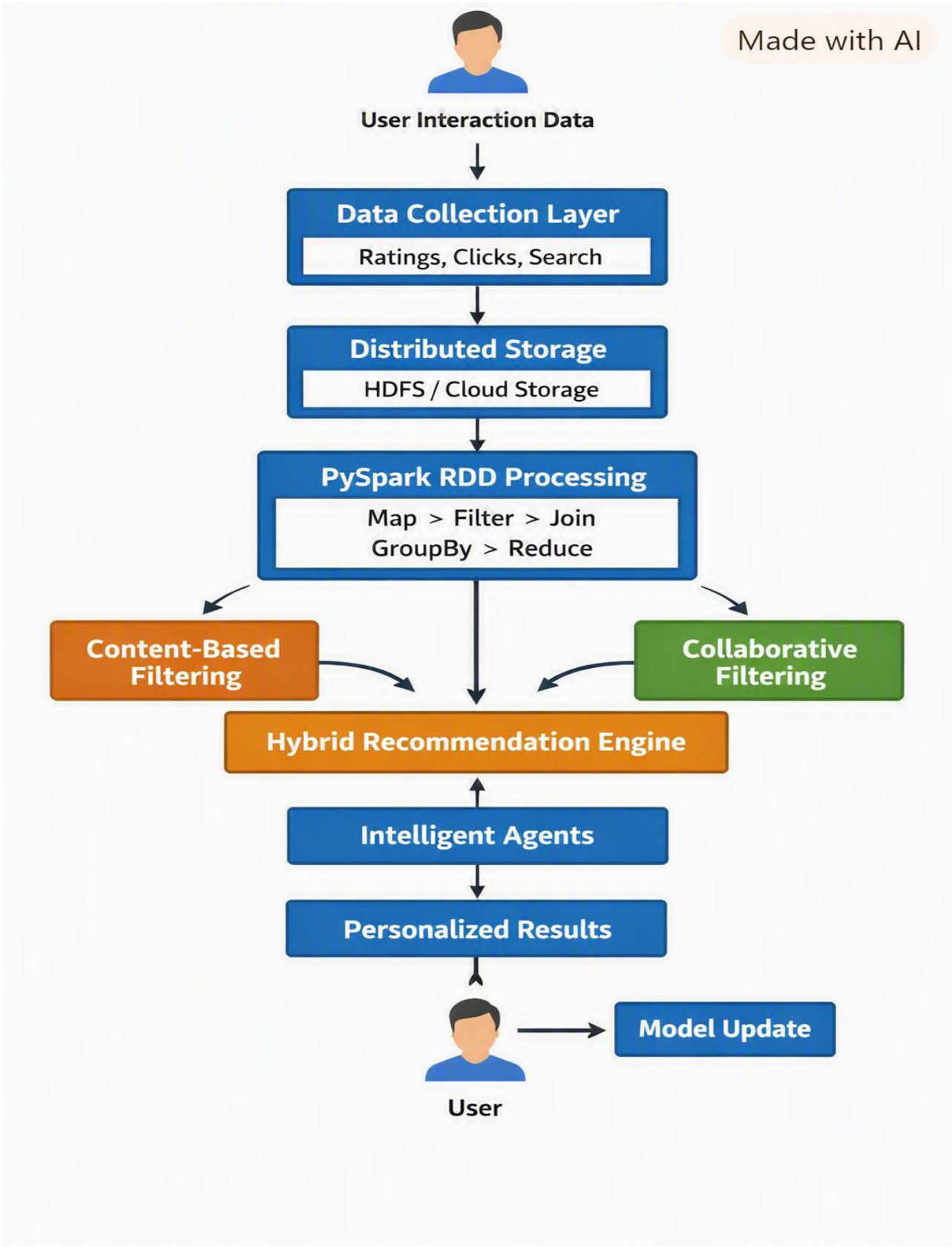
Objectives

- Provide personalized recommendations to users.
- Combine content-based and collaborative filtering.
- Process millions of user-item interactions using PySpark RDDs.
- Adapt recommendations according to real-time user behavior.
- Use intelligent agents to monitor user preferences and feedback.
- Provide scalable and fault-tolerant recommendation services.

Requirements

- User information and preference data.
- Item information such as category, keywords, and description.
- User-item ratings/interactions.
- Real-time feedback such as clicks, likes, purchases, and skips.
- Distributed processing using PySpark.

1.2 System Architecture



1.3 PySpark RDD Operations

RDDs provide distributed and fault-tolerant processing of large datasets.

Example user-item interaction RDD:

```
ratings = sc.textFile("ratings.csv")


data = ratings.map(lambda x: x.split(","))


user_item = data.map(
   lambda x: (x[0], (x[1], float(x[2])))
)


filtered = user_item.filter(
   lambda x: x[1][1] >= 3
)


grouped = filtered.groupByKey()


recommendations = grouped.mapValues(
   lambda items: list(items)
)


result = recommendations.collect()
```

Important RDD Operations

- **map()** – transforms each record.
- **filter()** – removes irrelevant records.
- **flatMap()** – generates multiple records from one input.
- **reduceByKey()** – combines values having the same key.
- **groupByKey()** – groups values according to keys.

- **join()** – combines two RDDs based on a common key.
- **distinct()** – removes duplicate records.
- **collect()** – retrieves results to the driver.

For example:

```
user_ratings = ratings.map(
   lambda x: (x.user_id, x.rating)
)


average_rating = user_ratings.reduceByKey(
   lambda a, b: a + b
)
```

1.4 Hybrid Recommendation Method

Content-Based Filtering

The system recommends items similar to items previously liked by the user.

Example:

User likes:

Machine Learning → Python → Artificial Intelligence

System recommends:

Deep Learning → Data Science → Neural Networks

Item features such as category, keywords, and descriptions are converted into feature vectors. Similarity between items can then be calculated.

Collaborative Filtering

Collaborative filtering identifies users with similar behavior.

User A → Item 1, Item 2, Item 3

User B → Item 1, Item 2

Since A and B have similar interests,

recommend Item 3 to User B.

Hybrid Score

The final recommendation score can be calculated as:

Hybrid Score = α(Content Score) + β(Collaborative Score)

where α and β represent the importance given to each method.

1.5 Intelligent Agent Integration

Different intelligent agents perform different responsibilities.

User Preference Agent

Monitors user behavior and maintains the user's preference profile.

Recommendation Agent

Generates personalized recommendations using content and collaborative scores.

Feedback Agent

Analyzes likes, dislikes, clicks, purchases, and skipped recommendations.

Adaptation Agent

Updates recommendation strategies based on recent user behavior.

Example

User clicks on AI courses

↓

Preference Agent detects interest

↓

Recommendation Agent increases AI-related content

↓

User provides feedback

↓

Feedback Agent analyzes response

↓

Adaptation Agent updates profile

1.6 Scalability and Distributed Computing

PySpark distributes RDD partitions across multiple worker nodes.

Advantages:

- Parallel processing.
- Fault tolerance through RDD lineage.
- Handles very large datasets.
- Faster processing compared with single-machine processing.
- Supports horizontal scaling.
- Data can be partitioned based on user IDs or item IDs.
- Frequently used data can be cached in memory.

1.7 Data Processing Pipeline

It shows:

- Data Collection → gathering user interactions
- Data Cleaning → removing noise and inconsistencies
- RDD Creation → forming distributed datasets in PySpark
- Feature Extraction → identifying relevant attributes
- Content Filtering and Collaborative Filtering → generating recommendations
- Hybrid Score Calculation → combining both filtering methods
- Agent-Based Adaptation → intelligent agents personalize results
- Personalized Recommendation → tailored suggestions for each user
- User Feedback → collecting responses
- Continuous Update → refining the model iteratively

1.8 Innovation

The system combines distributed processing with intelligent agents. Real-time behavioral feedback allows the recommendation engine to continuously adapt rather than producing static recommendations.

1.9 Conclusion

The proposed Hybrid AI Recommendation Engine provides accurate and personalized recommendations by combining content-based and collaborative filtering. PySpark RDDs enable distributed processing of large-scale interaction data, while intelligent agents continuously monitor behavior and feedback to improve recommendations.

2. Smart Disaster Detection System

Develop a distributed AI system using PySpark and RDD operations to detect natural disasters (such as floods or earthquakes) from multi-source data streams (satellite images, sensors, and social media). Describe the system architecture, data fusion techniques, and how intelligent agents coordinate to provide early warnings and response strategies.

2.1 Problem Understanding and Requirement Analysis

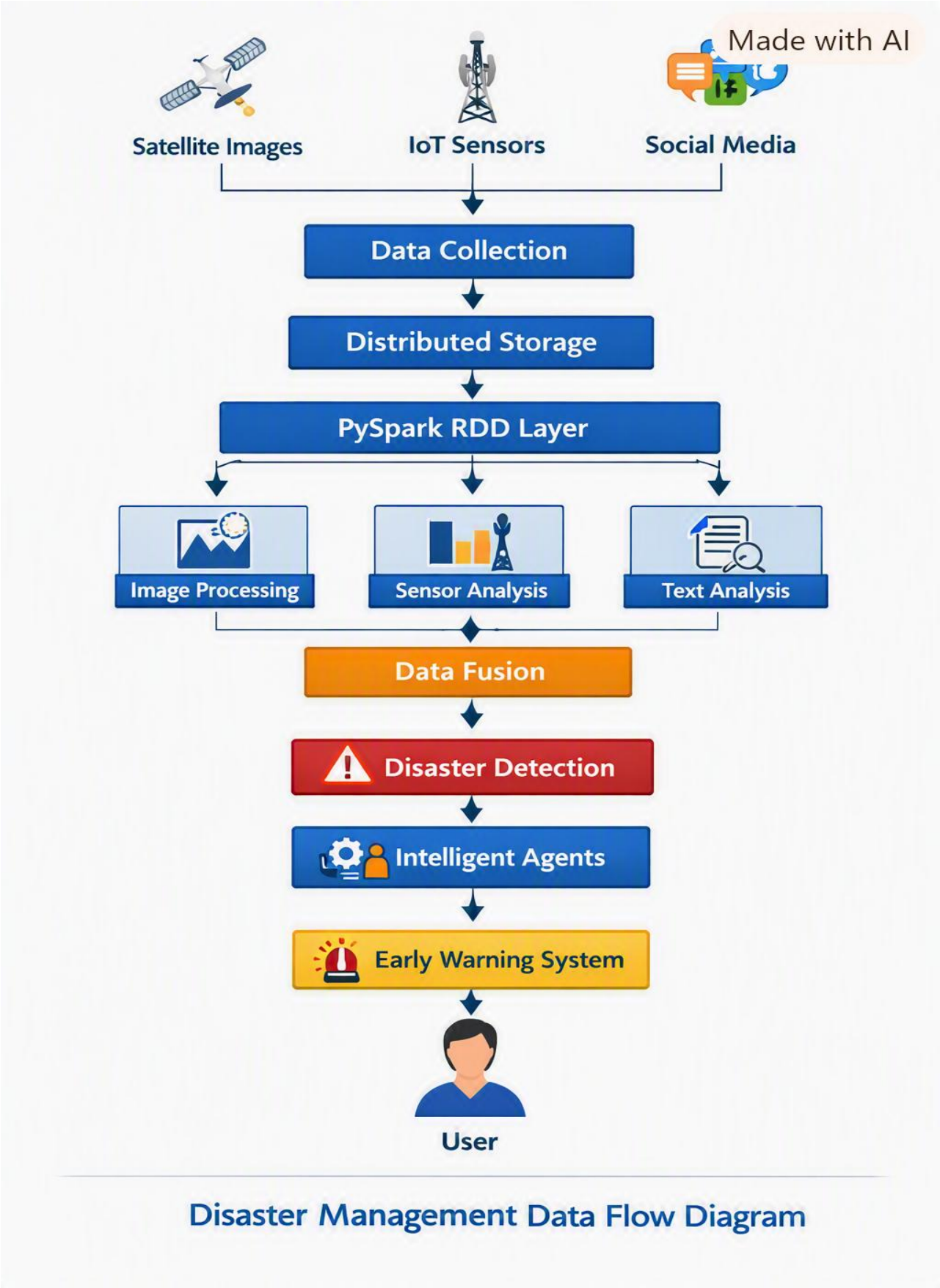
A Smart Disaster Detection System is a distributed AI system designed to detect natural disasters such as **floods, earthquakes, landslides, and cyclones** using multiple data sources.

The system processes satellite images, IoT sensor readings, weather information, and social media data to identify disaster conditions at an early stage.

Objectives

- Detect disasters automatically.
- Combine information from multiple sources.
- Process large-scale data using PySpark.
- Provide early warnings.
- Reduce false alarms through data fusion.
- Coordinate response activities using intelligent agents.

2.2 System Architecture



Disaster Management Data Flow Diagram

2.3 Data Sources

Satellite Data

Provides images for detecting flooded areas, damaged infrastructure, and changes in land surfaces.

IoT Sensors

Sensors provide:

- Water level.
- Temperature.
- Seismic activity.
- Rainfall.
- Pressure.

Social Media

Posts can provide real-time information such as:

"Heavy flooding near the bridge"

"Strong earthquake felt in the city"

2.4 PySpark RDD Processing

```
sensor_rdd = sc.textFile("sensor_data.csv")


data = sensor_rdd.map(
   lambda x: x.split(",")
)


high_water = data.filter(
   lambda x: float(x[2]) > 80
)


location_data = high_water.map(
```

```
    lambda x: (x[1], float(x[2]))
)

max_level = location_data.reduceByKey(
    lambda a, b: max(a, b)
)
```

Social media text can also be processed using RDDs:

```
posts = sc.textFile("social_media.txt")

disaster_words = [
    "flood", "earthquake",
    "landslide", "cyclone"
]

alerts = posts.filter(
    lambda x: any(word in x.lower()
                  for word in disaster_words)
)
```

2.5 Data Fusion

Data fusion combines information from different sources.

For example:

Satellite → Flooded region detected

Sensor → Water level extremely high

Social Media → People reporting flooding

Weather → Heavy rainfall

When multiple sources indicate the same event, the system increases the disaster confidence score.

Example:

Disaster Confidence = 0.35(Satellite) + 0.30(Sensor) + 0.20(Social Media) + 0.15(Weather)

If the confidence exceeds a predefined threshold, an alert is generated.

2.6 Intelligent Agent Integration

Sensor Monitoring Agent

Continuously monitors IoT sensors.

Satellite Analysis Agent

Analyzes satellite information and detects geographical changes.

Social Media Agent

Identifies disaster-related posts and emergency reports.

Data Fusion Agent

Combines information from different agents.

Alert Agent

Generates warnings when disaster probability becomes high.

Response Agent

Suggests actions such as evacuation, rescue-team deployment, and resource allocation.

2.7 Scalability

PySpark divides the incoming data into partitions and processes them simultaneously on worker nodes.

Benefits:

- Handles high-volume sensor data.
- Supports parallel image/text processing.
- Fault tolerance through RDD lineage.
- Can add more worker nodes when data volume increases.
- Supports continuous processing pipelines.

2.8 Data Flow

Multi-source Data

↓

Data Cleaning

↓

RDD Creation

↓

Parallel Processing

↓

Feature Extraction

↓

Data Fusion

↓

Disaster Probability

↓

Agent Coordination

↓

Early Warning

↓

Emergency Response

2.9 Conclusion

The Smart Disaster Detection System combines multi-source data, distributed PySpark processing, AI-based detection, and intelligent agents to provide early and reliable disaster warnings. The distributed architecture allows the system to operate efficiently even with massive real-time data.

3. **AI-Based Fraud Detection Framework**

Design a scalable fraud detection system using PySpark RDDs to process large volumes of financial transactions. Explain how anomaly detection algorithms can be implemented in a distributed manner and how intelligent agents collaborate to identify suspicious patterns, trigger alerts, and continuously improve detection accuracy.

3.1 Problem Understanding and Requirement Analysis

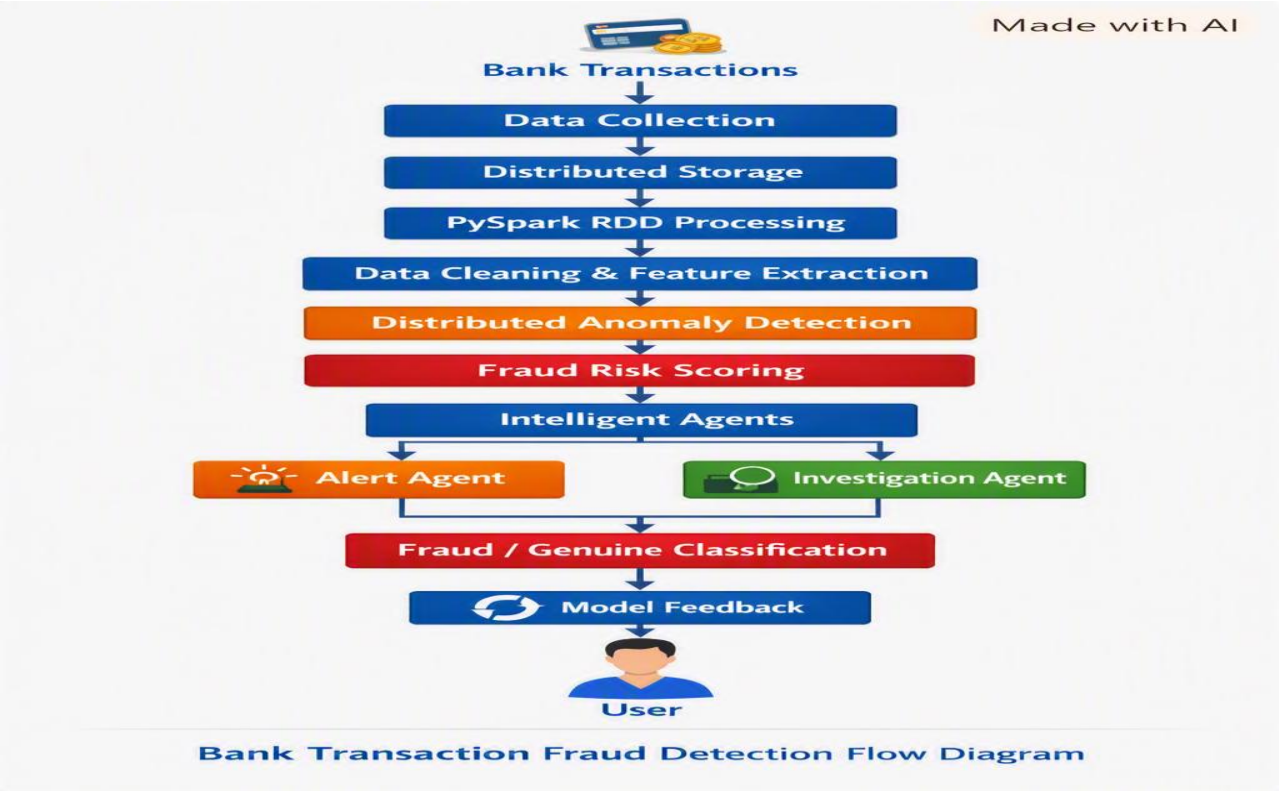
Financial institutions process millions of transactions every day. Traditional rule-based systems may fail to detect new and complex fraud patterns.

The proposed AI-Based Fraud Detection Framework uses **PySpark RDDs, anomaly detection, and intelligent agents** to identify suspicious transactions.

Objectives

- Process millions of transactions.
- Detect unusual transaction behavior.
- Identify suspicious patterns.
- Generate real-time alerts.
- Reduce false positives.
- Continuously improve detection accuracy.

3.2 System Architecture



3.3 RDD Operations

Transaction data may contain:

Transaction_ID

User_ID

Amount

Location

Time

Merchant

Device

Example:

```
transactions = sc.textFile("transactions.csv")


data = transactions.map(
   lambda x: x.split(",")
)


valid_data = data.filter(
   lambda x: float(x[2]) > 0
)


amounts = valid_data.map(
   lambda x: (x[1], float(x[2]))
)


total_amount = amounts.reduceByKey(
   lambda a, b: a + b
)
```

The reduceByKey() operation calculates the total transaction amount for each customer in a distributed manner.

3.4 Distributed Anomaly Detection

The system calculates behavioral features such as:

- Average transaction amount.
- Transaction frequency.
- Geographical distance.
- Login/device changes.
- Unusual transaction time.
- Number of failed transactions.

An anomaly score can be represented as:

$$\textbf{Anomaly Score} = w_1\text{Amount} + w_2\text{Frequency} + w_3\text{Location} + w_4\text{Time} + w_5\text{Device}$$

If the score exceeds a threshold, the transaction is marked as suspicious.

Example:

Normal transaction:

₹500 → Chennai → usual device → normal time

Suspicious transaction:

₹90,000 → different country → unknown device → unusual time

3.5 Intelligent Agent Integration

Transaction Monitoring Agent

Monitors transactions continuously.

Anomaly Detection Agent

Calculates fraud/anomaly scores.

Pattern Analysis Agent

Identifies repeated suspicious behavior.

Alert Agent

Generates alerts for high-risk transactions.

Investigation Agent

Collects evidence and supports manual investigation.

Learning Agent

Uses confirmed fraud cases as feedback to improve future detection.

3.6 Agent Collaboration

It shows:

- Transaction → Monitoring Agent → Anomaly Agent → Pattern Agent → Risk Score
- The Risk Score splits into two paths:
    - Low Risk → Normal
    - High Risk → Alert Agent → Investigation → Learning Agent

This flow captures how agents collaborate dynamically — from detecting anomalies and patterns to assessing risk, triggering alerts, and learning from investigations to improve future detection accuracy.

3.7 Scalability and Fault Tolerance

PySpark provides:

- Partition-based parallel processing.
- Distributed computation.
- In-memory processing.
- Fault recovery using RDD lineage.
- Horizontal scalability.
- Efficient processing of large transaction datasets.

Transactions can be partitioned by customer, geographical region, or time period.

3.8 Data Processing Pipeline

It includes:

- Transaction Data → Cleaning → RDD Creation → Feature Extraction
- Parallel Anomaly Detection → Risk Scoring → Agent-Based Analysis
- Fraud Alert → Investigation → Feedback → Model Improvement

3.9 Innovation

The system combines anomaly detection with multi-agent collaboration. Instead of depending only on fixed fraud rules, the learning agent continuously uses feedback from confirmed fraud cases to improve detection.

3.10 Conclusion

The proposed fraud detection framework is scalable, adaptive, and fault tolerant. PySpark enables large-scale transaction processing, while intelligent agents collaborate to detect suspicious patterns, generate alerts, and continuously improve the system.

4. **Autonomous Supply Chain Optimization System**

Construct a distributed AI system using PySpark for optimizing supply chain operations (inventory, logistics, demand forecasting). Describe how RDD transformations support large-scale data processing and how multiple intelligent agents interact to make decentralized decisions for cost minimization and efficiency improvement.

4.1 Problem Understanding and Requirement Analysis

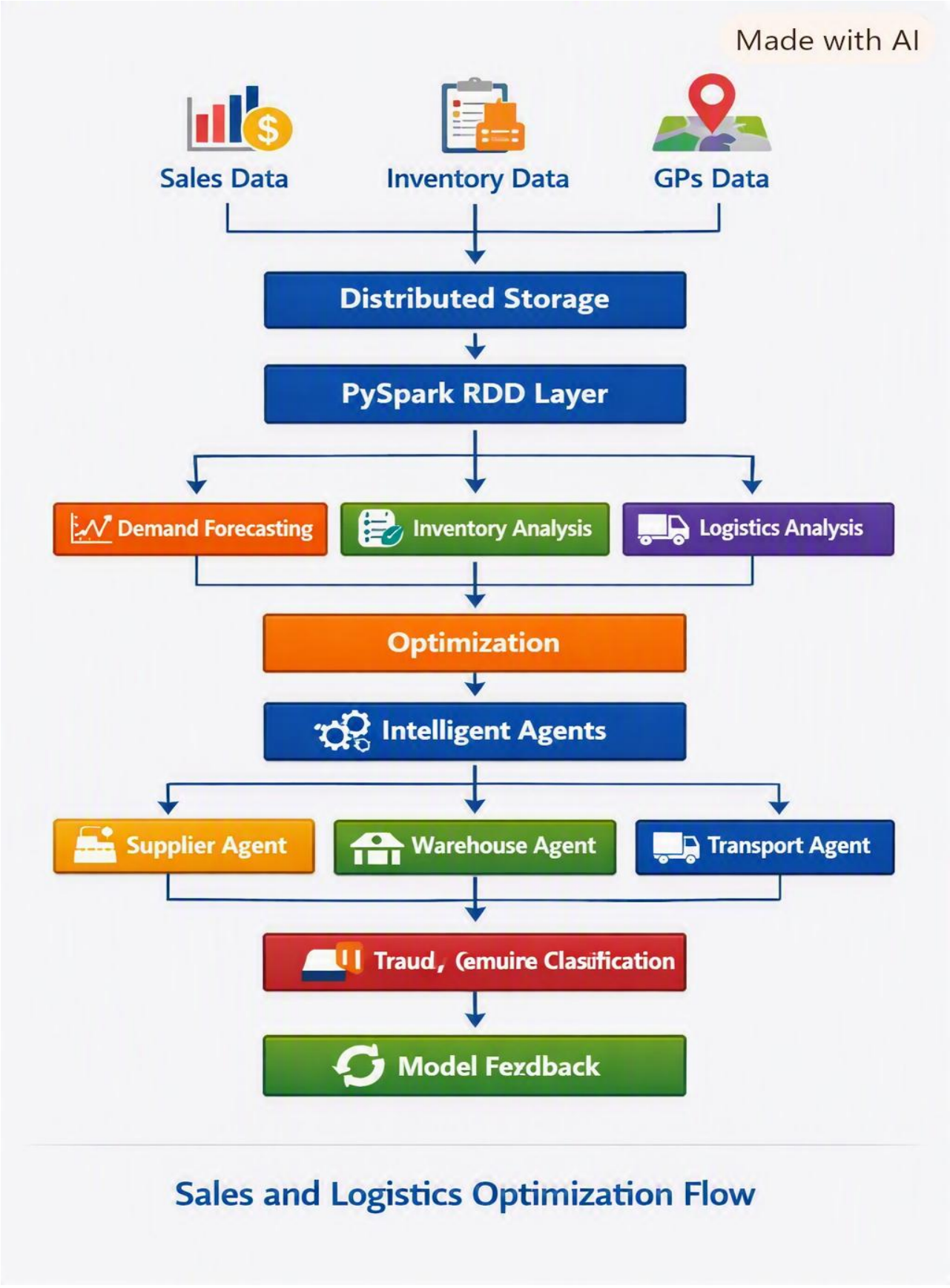
Supply chains involve inventory management, transportation, suppliers, warehouses, and demand forecasting. Large organizations generate huge volumes of data that must be processed efficiently.

The proposed system uses **PySpark RDDs and intelligent agents** to optimize inventory, logistics, and demand forecasting.

Objectives

- Predict product demand.
- Maintain optimal inventory levels.
- Optimize transportation routes.
- Reduce operational costs.
- Avoid stock-outs and overstocking.
- Enable decentralized intelligent decision-making.

4.2 System Architecture



4.3 PySpark RDD Operations

```
sales = sc.textFile("sales.csv")


data = sales.map(
   lambda x: x.split(",")
)


valid_sales = data.filter(
   lambda x: float(x[2]) > 0
)


product_sales = valid_sales.map(
   lambda x: (x[1], float(x[2]))
)


total_sales = product_sales.reduceByKey(
   lambda a, b: a + b
)
```

RDD Transformations Used

- map() – transforms sales records.
- filter() – removes invalid records.
- flatMap() – processes multiple product records.
- reduceByKey() – calculates total sales.
- groupByKey() – groups sales by product.
- join() – combines sales with inventory data.
- sortByKey() – sorts products based on selected criteria.

4.4 Demand Forecasting

Historical sales data is processed to identify demand patterns.

Historical Sales

↓

RDD Processing

↓

Daily/Weekly Demand

↓

Demand Prediction

↓

Inventory Decision

If predicted demand is high, the system can automatically increase inventory orders.

4.5 Intelligent Agent Integration

Supplier Agent

Selects suppliers based on price, reliability, and delivery time.

Inventory Agent

Monitors stock levels and determines reorder requirements.

Logistics Agent

Optimizes transportation and delivery decisions.

Demand Forecasting Agent

Predicts future product demand.

Warehouse Agent

Manages warehouse capacity and product movement.

Coordination Agent

Coordinates decisions among all agents.

4.6 Decentralized Decision Making

Example:

Demand Agent → High demand predicted

↓

Inventory Agent → Stock is insufficient

↓

Supplier Agent → Selects suitable supplier

↓

Logistics Agent → Selects efficient transport

↓

Warehouse Agent → Allocates storage

This reduces dependency on a single centralized decision-maker.

4.7 Cost Optimization

The system attempts to minimize:

Total Cost = Inventory Cost + Transportation Cost + Ordering Cost + Stock-out Cost

The agents compare different decisions and select the option with minimum expected cost while maintaining service quality.

4.8 Scalability

PySpark distributes supply-chain data across worker nodes.

Advantages:

- Processes large sales datasets.
- Handles multiple warehouses and regions.
- Supports parallel demand analysis.
- Fault tolerant.
- Easy to scale by adding worker nodes.
- Supports data partitioning by product or region.

4.9 Data Flow

Sales + Inventory + Logistics Data

↓

Data Cleaning

↓

RDD Creation

↓

Distributed Processing

↓

Demand + Inventory + Route Analysis

↓

Intelligent Agents

↓

Decision Optimization

↓

Supplier/Warehouse/Transport

↓

Continuous Feedback

4.10 Innovation

The system uses autonomous agents to make decentralized supply-chain decisions. Each agent specializes in a particular operation while coordinating with other agents to minimize overall cost.

4.11 Conclusion

The Autonomous Supply Chain Optimization System combines PySpark's distributed processing capabilities with AI-based forecasting and intelligent agents. It can reduce inventory costs, improve logistics efficiency, prevent stock-outs, and support large-scale supply-chain operations.

5. Personalized Learning Analytics Platform

Design an AI-driven educational platform that uses PySpark and RDDs to analyze student learning behavior and performance data. Explain how the system supports adaptive learning through intelligent agents, real-time feedback, and scalable analytics to personalize content delivery for thousands of learners.

5.1 Problem Understanding and Requirement Analysis

A Personalized Learning Analytics Platform analyzes student learning behavior and academic performance to provide customized learning content.

The platform processes data such as:

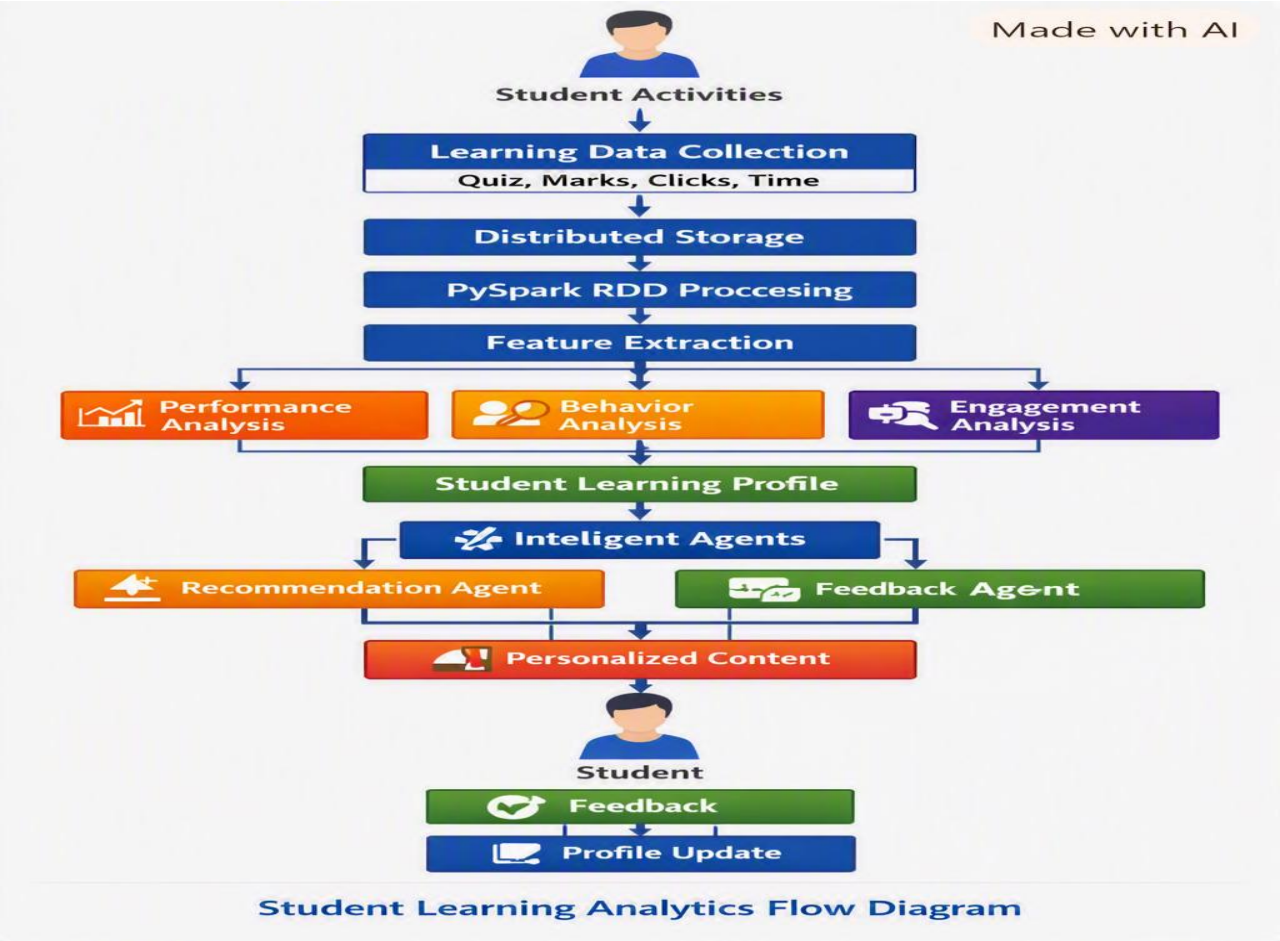
- Quiz scores.
- Assignment marks.
- Attendance.

- Course completion.
- Video watching behavior.
- Time spent learning.
- Learning preferences.
- Student feedback.

Objectives

- Analyze student learning behavior.
- Identify weak and strong topics.
- Provide personalized learning materials.
- Give real-time feedback.
- Predict student performance.
- Support thousands of learners simultaneously.

5.2 System Architecture



5.3 PySpark RDD Processing

Student data can be loaded into an RDD:

```
students = sc.textFile("student_data.csv")

data = students.map(
   lambda x: x.split(",")
)

valid_data = data.filter(
   lambda x: float(x[2]) >= 0
)

student_scores = valid_data.map(
   lambda x: (x[0], float(x[2]))
)

total_scores = student_scores.reduceByKey(
   lambda a, b: a + b
)
```

Average performance can be calculated using:

```
score_data = student_scores.mapValues(
   lambda x: (x, 1)
)

result = score_data.reduceByKey(
   lambda a, b: (a[0] + b[0], a[1] + b[1])
)
```

5.4 Learning Analytics

The platform calculates features such as:

- Average quiz score.
- Number of completed lessons.
- Time spent on each topic.
- Number of attempts.
- Frequently incorrect questions.
- Course completion rate.

Example:

Student A

Mathematics → 85%

Programming → 45%

AI → 90%

The system identifies Programming as a weak area.

5.5 Personalized Recommendation

Based on the student's performance:

Low Programming Score

↓

Learning Agent detects weakness

↓

Recommendation Agent

↓

Suggests:

• Basic Programming Videos

• Practice Questions

• Interactive Exercises

• Revision Materials

For a high-performing student, advanced materials can be recommended.

5.6 Intelligent Agent Integration

Student Monitoring Agent

Tracks student activity and performance.

Performance Analysis Agent

Identifies strong and weak areas.

Recommendation Agent

Selects suitable learning materials.

Feedback Agent

Provides immediate feedback after quizzes and assignments.

Motivation Agent

Encourages students when engagement decreases.

Adaptation Agent

Changes learning difficulty according to student progress.

5.7 Real-Time Adaptive Learning

The system continuously observes student activity.

Student attempts quiz

↓

Score generated

↓

Performance Agent analyzes result

↓

Weak topic identified

↓

Recommendation Agent suggests content

↓

Student studies content

↓

New quiz

↓

Profile updated

Thus, the learning path changes dynamically according to student behavior.

5.8 Scalability and Distributed Computing

PySpark allows thousands or millions of student records to be processed in parallel.

Advantages:

- Distributed student-data processing.
- Parallel analytics.
- Fault tolerance.
- Horizontal scalability.
- Efficient processing of large educational datasets.
- RDD caching can improve repeated analytics.

Data can be partitioned by student ID, course, or institution.

5.9 Data Processing Pipeline

Student Activity Data

↓

Data Collection

↓

Data Cleaning

↓

RDD Creation

↓

Feature Extraction

↓

Performance Analysis

↓

Learning Profile

↓

Intelligent Agents

↓

Content Recommendation

↓

Real-Time Feedback

↓

Student Interaction

↓

Profile Update

5.10 Innovation

The platform provides an adaptive learning experience instead of giving the same content to every student. Intelligent agents continuously monitor progress and modify recommendations according to individual learning needs.

5.11 Conclusion

The Personalized Learning Analytics Platform combines PySpark RDD-based distributed analytics with intelligent agents to provide scalable and adaptive education. It can analyze thousands of learners, identify their strengths and weaknesses, recommend suitable content, and provide continuous feedback.

OVERALL CONCLUSION

All five proposed systems demonstrate how **PySpark RDDs, distributed computing, AI techniques, and intelligent agents** can be integrated to develop scalable AI applications.

PySpark RDDs provide:

- Distributed processing.
- Parallel execution.
- Fault tolerance.
- Scalability.
- Efficient processing of large datasets.

Intelligent agents provide:

- Autonomous decision-making.
- Continuous monitoring.

- Adaptation to changing conditions.
- Collaboration between specialized components.
- Real-time response and feedback.

Therefore, integrating **PySpark + RDD operations + intelligent agents** enables the development of scalable, adaptive, and intelligent distributed AI systems for real-world applications.

RUBRICS FOR CALCULATION

Evaluation Criteria	Problem Understanding & Requirement Analysis	System Architecture Design	Use of PySpark & RDD Operations	Intelligent Agent Integration	Scalability & Distributed Computing Design	Data Flow & Processing Pipeline	Innovation & Creativity	Clarity, Presentation & Documentation
Weightage	10%	20%	15%	15%	10%	10%	10%	10%

Criteria	Excellent (5)	Good (4)	Average (3)	Poor (2)
Problem Understanding & Requirement Analysis	Clearly defines problem, objectives, and constraints	Mostly clear with minor gaps	Basic understanding	Unclear or incorrect
System Architecture Design	Complete, scalable, well-structured architecture with diagrams	Good design with minor issues	Basic design, lacks detail	Poor/incomplete
Use of PySpark & RDD Operations	Efficient and correct use of RDD transformations/actions	Mostly correct usage	Limited or partial use	Incorrect/missing
Intelligent Agent Integration	Strong integration with clear agent roles and logic	Good integration	Basic integration	Poor/no integration
Scalability & Distributed Computing Design	Clearly addresses scalability, fault tolerance, parallelism	Covers most aspects	Limited consideration	Not addressed
Data Flow & Processing Pipeline	Clear, logical, and well-defined data flow	Mostly clear	Some gaps	Poorly defined

Innovation & Creativity	Highly innovative, realistic, and impactful solution	Some innovation	Basic idea	No originality
Clarity, Presentation & Documentation	Very clear, neat diagrams, well-organized report	Mostly clear	Somewhat organized	Poor presentation